

实验三：用户态线程（协程）参考手册

1. 协程的基本概念

1.1 什么是协程？

协程 (Coroutine) 是一种用户态的轻量级线程。与操作系统的内核线程 (Kernel Thread) 不同，协程的切换不依赖于内核的中断机制，而是由程序员通过代码显式地控制。

- **协作式调度**：协程必须主动利用 `yield` 让出 CPU 才能进行切换，不存在强制性的“抢占”（除非使用特殊的信号机制模仿）。
- **轻量级**：一个线程可以包含成千上万个协程，它们的开销极小。

1.2 为什么要用协程？

理解“为什么需要协程”，要先看传统模型的局限。

- **多线程并发的成本较高**：如果每个任务都开一个线程，线程数一多，管理成本就会明显上升。即使每个线程做的事情都很简单，系统也要为每个线程维护独立栈、调度状态和切换现场。

例如下面这段伪代码：

```
for (int i = 0; i < 10000; i++) {  
    pthread_create(&tid[i], NULL, worker, NULL);  
}
```

这里每个 `worker` 也许只是“收一个请求、等一下数据、再回一个结果”，但系统已经先创建了 10000 个线程。问题不在某个线程本身有多复杂，而在“任务数”和“线程数”被直接绑定了。

- **阻塞式 I/O 会浪费执行机会**：线程调用阻塞 I/O 后，会一直停在那里等结果返回。在等待期间，它不是在执行计算，只是在睡眠等待内核把它唤醒。

例如：

```
void worker(int fd) {  
    char buf[1024];  
    read(fd, buf, sizeof(buf));    // 数据没到就会阻塞  
    process(buf);  
}
```

如果 `read()` 还拿不到数据，这个线程就卡在这里。此时：

- `process(buf)` 还没有开始执行；
- 这个线程也不能去处理别的连接；
- 如果每个连接都对应一个线程，那么很多线程都可能只是停在 `read()` 上等待。

协程想解决的不是“等待时完全不占任何内存”，而是“不要让一个真正的线程专门傻等这件事”。更理想的做法是：当前任务先挂起，让底层线程去执行别的任务，等数据到了再恢复这个协程。

- **异步回调代码容易把一个顺序逻辑拆碎**：这里先解释一下“异步回调”是什么意思。所谓异步，就是“先发起一个操作，但不原地等待它做完”；所谓回调，就是“等这个操作完成后，再自动调用你提前注册好的函数”。很多

I/O 程序本来想表达的是“先读请求，再查数据，再写响应”，但如果完全用这种“做完一步就回调下一个函数”的方式组织，原本一条直线的控制流会被拆散。

例如：

```
void on_request(int fd) {
    async_read(fd, on_read_done);
}

void on_read_done(int fd, char *buf) {
    async_query_db(buf, on_db_done);
}

void on_db_done(int fd, Result *r) {
    async_write(fd, r, on_write_done);
}
```

这段代码当然能工作，但“读请求 -> 查数据库 -> 响应”这个流程被拆成了多个分散的函数。逻辑一长以后，状态传递、错误处理和调试都会变麻烦。

协程希望保留下面这种更接近顺序思维的写法：

```
void handle(int fd) {
    Request req = co_read(fd);
    Result r = co_query_db(req);
    co_write(fd, r);
}
```

表面上它像普通顺序代码，但在 `co_read`、`co_query_db` 这些点上，运行时可以先挂起当前协程，等条件满足后再恢复。

协程提供了一种折中方案：

- **比线程更轻**：协程切换通常只是在用户态保存和恢复上下文，不必每次都进入内核。
- **更适合高并发 I/O 场景**：大量任务大部分时间都在“等待”时，可以用少量线程承载大量协程。
- **保留顺序代码的写法**：虽然底层是“执行一段、挂起、再恢复”，但代码表面上仍然像普通函数一样线性书写。

可以把它理解为：线程解决的是“操作系统如何同时调度多个执行流”，而协程更关注“程序员如何在一个线程内组织大量会暂停和恢复的任务”。

典型适用场景包括：

- **网络服务器**：每个连接都可能频繁等待 I/O，协程能显著减少线程数量。
- **事件驱动程序**：例如聊天服务、爬虫、代理、游戏服务器。
- **需要大量并发但单任务计算不重的程序**：此时瓶颈更多在等待，而不是 CPU 算力。

但协程也不是万能的：

- **不能自动利用多核**：多个协程如果都跑在同一个线程里，本质上仍然只占用一个 CPU 核心。
- **不适合纯 CPU 密集型任务**：如果某个协程长时间不 `yield`，其他协程就会被“饿住”。
- **需要程序员理解调度点**：哪里可能挂起、哪里会恢复，必须在设计时想清楚。

因此，本实验选择协程，不是为了替代线程，而是为了让你亲手理解一种在现代高并发系统中非常常见的执行流组织方式。

1.3 核心构成：上下文（Context）

在协程切换中，最关键的是保存和恢复“状态”，这就是 **上下文**。它可以理解为一份“让 CPU 能从某个位置继续跑起来”的执行流快照。

当协程主动 `yield` 时，程序需要保存它当前执行到哪里、当前栈顶在哪里、寄存器里有哪些临时数据；之后再次 `resume` 时，再把这些状态恢复回来，于是协程就能像从未离开过一样继续执行。

在本实验中，这份上下文由 POSIX 的 `ucontext_t` 结构体表示。下一节会具体说明 `ctx` 中有哪些字段，以及代码模板为什么要设置 `uc_stack.ss_sp`、`uc_stack.ss_size`、`uc_stack.ss_flags` 和 `uc_link`。

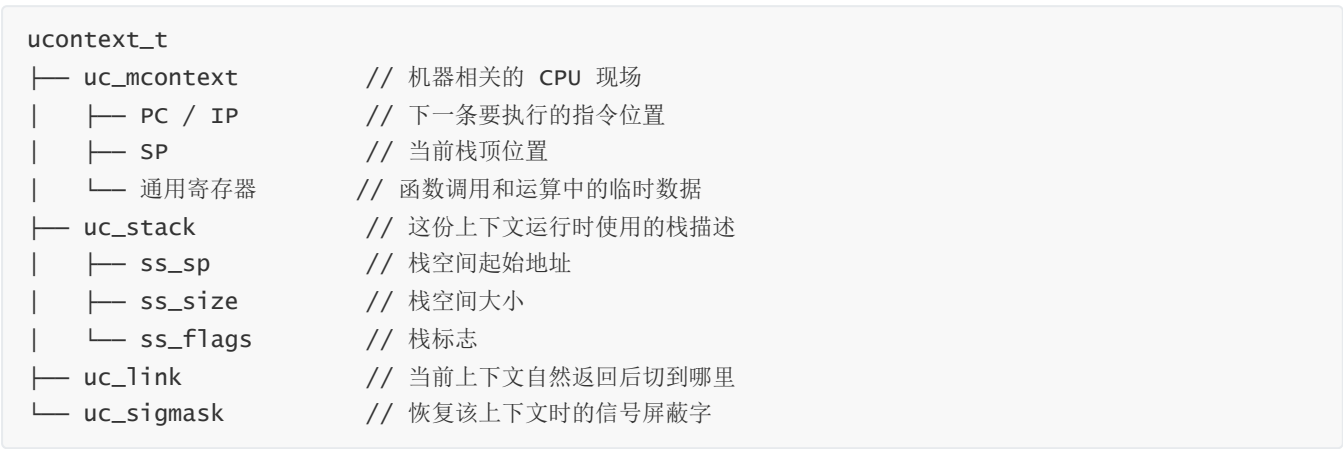
2. 理解 `<ucontext.h>` API

POSIX 提供的 `ucontext` 库是实现用户态线程最常见的工具。它定义了一个核心结构体 `ucontext_t`。

2.1 `ucontext_t` 结构体

`ucontext_t` 可以理解为上下文的具体存储格式。它既保存 CPU 执行现场，也保存这份上下文运行所需的元信息。

如果先从整体上看，可以把它理解成下面这样的逻辑结构：



注意：上面这张树是“帮助理解的逻辑结构图”，不是要求你死记平台相关源码定义。不同系统里 `ucontext_t` 和 `mcontext_t` 的内部细节可能不完全一样，但从实验角度看，抓住这几部分就够了。

CPU 执行现场

- `uc_mcontext`：保存 CPU 寄存器状态的部分，通常包括 PC/IP、SP 和通用寄存器等。实验代码一般不直接修改它，而是交给 `getcontext`、`makecontext`、`swapcontext` 和 `setcontext` 处理。
- **PC/IP (Program Counter / Instruction Pointer)**：下一条要执行的指令地址。第一次运行协程时，它会指向 `coroutine_wrapper`；协程 `yield` 后再恢复时，它会指向上次暂停后应该继续执行的位置。
- **SP (Stack Pointer)**：当前栈顶的位置。它不是简单等于 `stack` 数组首地址，而是会随着函数调用、返回、局部变量压栈而变化。
- **通用寄存器**：保存正在运算的数据、函数调用过程中的临时状态等。

上下文运行元信息

- `uc_stack`：该上下文运行时使用的栈描述。
- `uc_stack.ss_sp`：栈空间起始地址。本实验中通常设置为 `co->stack`。
- `uc_stack.ss_size`：栈空间大小。本实验中通常设置为 `STACK_SIZE`。
- `uc_stack.ss_flags`：栈的标志位，普通可用栈一般设置为 `0`。
- `uc_link`：当前上下文的入口函数自然返回后，自动切换到哪个上下文。本实验中可以设置为 `&scheduler.main_ctx`，作为协程结束后的返回去向。
- `uc_sigmask`：恢复该上下文时使用的信号屏蔽字。基础协程部分通常不需要手动修改；做抢占式调度、信号处理时才更容易遇到它。

因此，代码模板中的 `co->ctx` 不只是“寄存器备份”，还记录了“这个协程应该使用哪块栈、栈有多大、自然结束后去哪里”。

2.2 核心函数

函数	作用
<code>getcontext(ucontext_t *ucp)</code>	将当前 CPU 的寄存器状态保存到 <code>ucp</code> 中。
<code>setcontext(const ucontext_t *ucp)</code>	将 CPU 状态恢复为 <code>ucp</code> 中的样子（此函数不返回，直接跳转）。
<code>makecontext(ucontext_t *ucp, void (*func)(), int argc, ...)</code>	在已有上下文基础上，修改 <code>ucp</code> 的入口位置和启动参数，使其第一次被恢复时从 <code>func</code> 开始执行。
<code>swapcontext(ucontext_t *oucp, const ucontext_t *ucp)</code>	原子切换 ：保存当前状态到 <code>oucp</code> ，并立即切换到 <code>ucp</code> 执行。

3. 调用栈（Call Stack）的隔离

每个协程都需要一块独立的内存作为其运行时的“栈”。

- **局部变量存储**：当协程函数调用其他子函数时，其参数和局部变量都压在这一块申请的内存里。
- **栈溢出（Stack Overflow）**：如果在这个内存里分配了过大的局部数组或发生了深度递归，就会覆盖掉相邻的内存区域，导致程序崩溃。

3.1 `stack`、`uc_stack` 和栈顶 `SP` 到底是什么关系？

这三个概念很容易混在一起，但它们的职责并不相同：

- `char stack[STACK_SIZE]`：真正的栈内存本体。局部变量、返回地址、函数调用现场都存放在这里。
- `ctx.uc_stack`：对这块栈内存的描述信息，告诉 `ucontext` 运行时“这块栈在哪里、有多大”。
- **SP（Stack Pointer）**：当前栈顶指针，表示“这块栈现在用到哪了”。它保存在上下文寄存器状态里，会随着函数调用和返回不断变化。

所以：

- `stack` 的地址通常是固定的；

- `uc_stack` 记录的是这块栈的边界信息；
- 真正会在运行时不停变化的是 `SP`，也就是“当前栈顶的位置”。

3.2 局部变量为什么不会在协程切换后消失？

因为协程切换时，保存的不只是“程序执行到哪一行”，还包括“当前用的是哪块栈、栈顶停在哪个位置”。

当一个协程执行到一半调用 `yield()` 时：

- 该协程当前的寄存器状态会被保存到自己的 `ctx` 中；
- 其中包括当前的 `PC`、`SP` 等关键寄存器；
- 该协程栈上的局部变量仍然留在它自己的 `stack` 里，没有被清空。

下一次 `resume()` 它时：

- `swapcontext` 会把这个协程之前保存的 `SP` 恢复回来；
- CPU 继续在同一块协程栈上执行；
- 因此函数里的局部变量还能保持之前的值。

这也是协程“暂停后还能接着跑”的根本原因。

3.3 多个协程同时存在时，内存上是什么样子？

对于实验中的两个协程 `co1` 和 `co2`，可以把内存关系粗略理解成这样：

```
Scheduler
├─ main_ctx           // 主调度器上下文
├─ coroutines[0]
│  ├─ ctx             // 协程 0 的寄存器现场
│  ├─ stack[64KB]     // 协程 0 的独立栈
│  ├─ state
│  └─ func
└─ coroutines[1]
   ├─ ctx             // 协程 1 的寄存器现场
   ├─ stack[64KB]     // 协程 1 的独立栈
   ├─ state
   └─ func
```

注意：

- 两个协程的 `stack` 是两块不同的内存，不共享；
- 两个协程的 `ctx` 也是分别保存的；
- 切换协程时，不会搬动它们的内存位置，只是 CPU 改为加载不同协程保存下来的寄存器状态。

4. 多协程运行时，关键变量是如何变化的？

下面用一个最小例子说明两个协程交替运行时，`state`、`current_id`、局部变量和上下文是如何变化的。

4.1 示例代码

```
void task1() {
    int x = 100;
    printf("task1: x = %d\n", x);
    coroutine_yield();
    x++;
    printf("task1: x = %d\n", x);
}

void task2() {
    int y = 200;
    printf("task2: y = %d\n", y);
    coroutine_yield();
    y += 2;
    printf("task2: y = %d\n", y);
}
```

假设：

- task1 对应 coroutines[0]
- task2 对应 coroutines[1]
- 调度器采用最简单的轮流 resume(0)、resume(1) 的方式

4.2 创建完成后的状态

调用：

```
scheduler_init();
coroutine_spawn(task1);
coroutine_spawn(task2);
```

此时：

变量/对象	状态
scheduler.current_id	-1
coroutines[0].state	RUNNABLE
coroutines[1].state	RUNNABLE
coroutines[0].ctx	已经设置好入口函数、独立栈、uc_link
coroutines[1].ctx	已经设置好入口函数、独立栈、uc_link
task1 的局部变量 x	还不存在，因为 task1 还没真正开始执行
task2 的局部变量 y	还不存在，因为 task2 还没真正开始执行

关键点：

- `spawn` 只是把协程“准备好”;
- 协程函数本身还没开始跑;
- 所以函数内局部变量此时还没有入栈。

4.3 第一次 `resume(0)` 之后发生什么

调度器执行:

```
coroutine_resume(0);
```

这时:

1. `coroutines[0].state` 从 `RUNNABLE` 变为 `RUNNING`
2. `scheduler.current_id` 从 `-1` 变为 `0`
3. `swapcontext(&main_ctx, &coroutines[0].ctx)` 保存主调度器现场
4. CPU 开始执行 `task1`

当 `task1` 跑到:

```
int x = 100;
```

时, 变化如下:

- `x` 被压入 **协程 0 的栈** `coroutines[0].stack`
- 当前 `SP` 指向协程 0 的栈内某个位置
- 协程 1 的栈完全没动

随后 `task1` 调用 `coroutine_yield()`:

- `coroutines[0].state` 变成 `SUSPEND`
- `scheduler.current_id` 变回 `-1`
- `swapcontext(&coroutines[0].ctx, &main_ctx)` 把协程 0 当前的 `PC`、`SP` 等现场保存到 `ctx`
- `x=100` 仍然留在协程 0 的栈里

此时可以总结成:

变量/对象	状态
<code>scheduler.current_id</code>	<code>-1</code>
<code>coroutines[0].state</code>	<code>SUSPEND</code>
<code>coroutines[1].state</code>	<code>RUNNABLE</code>
<code>task1</code> 的 <code>x</code>	已存在于 <code>coroutines[0].stack</code> 中, 值为 <code>100</code>
<code>task2</code> 的 <code>y</code>	还不存在

4.4 再执行 `resume(1)` 时会怎样

调度器执行：

```
coroutine_resume(1);
```

这时：

- CPU 切到协程 1 的上下文
- `coroutines[1].state = RUNNING`
- `scheduler.current_id = 1`
- `task2` 开始执行

当 `task2` 跑到：

```
int y = 200;
```

时：

- `y` 被压入 **协程 1 的栈** `coroutines[1].stack`
- 协程 0 栈里的 `x=100` 仍然原封不动
- 两个协程的局部变量分别待在各自的栈里，互不影响

当 `task2` 也执行 `yield()` 后：

变量/对象	状态
<code>scheduler.current_id</code>	<code>-1</code>
<code>coroutines[0].state</code>	<code>SUSPEND</code>
<code>coroutines[1].state</code>	<code>SUSPEND</code>
<code>task1</code> 的 <code>x</code>	在协程 0 栈中，值为 <code>100</code>
<code>task2</code> 的 <code>y</code>	在协程 1 栈中，值为 <code>200</code>

4.5 再次 `resume(0)` 时，为什么能接着上次继续？

调度器再次执行：

```
coroutine_resume(0);
```

恢复后，CPU 会使用协程 0 上次保存下来的 `ctx`：

- PC 恢复到 `yield()` 返回之后的位置
- SP 恢复到协程 0 上次暂停时的栈顶

因此 `task1` 会继续执行：


```
x++;
printf("task1: x = %d\n", x); // 打印 101
```

这里的 `x` 之所以还能从 100 变成 101，是因为：

- `x` 一直在协程 0 自己的栈里；
- 之前切走时只是暂停，没有清空；
- 恢复时又回到了同一个栈顶位置继续执行。

4.6 一个总表：什么不变，什么在变

项目	是否变化	说明
<code>coroutines[i].stack</code> 的地址	通常不变	协程创建后，这块栈内存的位置通常固定
<code>coroutines[i].ctx</code> 存放的位置	通常不变	上下文对象一直在该协程控制块里
栈里的内容	会变化	局部变量、返回地址、调用现场会不断压栈/出栈
<code>SP</code>	会变化	随函数调用、返回和 <code>yield</code> 变化
<code>PC</code>	会变化	表示当前执行到了哪一条指令
<code>state</code>	会变化	<code>RUNNABLE</code> 、 <code>RUNNING</code> 、 <code>SUSPEND</code> 、 <code>FREE</code> 会切换
<code>current_id</code>	会变化	表示 CPU 当前正在运行哪个协程

这张表可以帮助你区分两件事：

- **内存位置是否固定**
- **运行时内容是否变化**

很多初学者以为“协程切换”意味着把整个协程搬来搬去，其实不是。真正发生的是：

- 协程自己的栈和控制块一直在那里；
- CPU 只是轮流读取不同协程保存的寄存器现场。

4.7 `swapcontext` 前后的一张简图

下面这张图把“主调度器切到协程 0，再由协程 0 `yield` 回主调度器”的过程画出来。

第一次 `resume(0)` 之前

CPU 当前寄存器

- └─ PC -> `main()` / 调度循环中的 `coroutine_resume(0)`
- └─ SP -> 主线程自己的栈

内存中的数据

- └─ `scheduler.main_ctx` // 还没写入最新现场，稍后会由 `swapcontext` 更新
- └─ `coroutines[0].ctx`
 - └─ PC -> `coroutine_wrapper`
 - └─ SP -> 协程 0 栈的初始位置
 - └─ `uc_stack` -> [`coroutines[0].stack`, `STACK_SIZE`]
- └─ `coroutines[0].stack` // 已分配，但 `task1` 的局部变量还没入栈

执行 `swapcontext(&main_ctx, &coroutines[0].ctx)` 之后

CPU 当前寄存器

- └─ PC -> `coroutine_wrapper` / `task1`
- └─ SP -> `coroutines[0].stack` 中当前栈顶

内存中的数据

- └─ `scheduler.main_ctx`
 - └─ PC -> 调度器里“`swapcontext` 返回之后”的位置
 - └─ SP -> 主线程自己的栈顶
- └─ `coroutines[0].ctx` // 仍然保存协程 0 的上下文描述
- └─ `coroutines[0].stack`
 - └─ `task1` 的局部变量
 - └─ 返回地址
 - └─ 调用现场

此时的关键理解是：

- `main_ctx` 里保存了“将来回到主调度器时应该从哪里继续”；
- CPU 已经开始使用协程 0 的栈，所以 `task1` 的局部变量会进入 `coroutines[0].stack`。

当协程 0 调用 `yield()` 时

执行的是：

```
swapcontext(&coroutines[0].ctx, &scheduler.main_ctx);
```

效果可以理解为：

步骤 1：把协程 0 当前现场写回 `coroutines[0].ctx`

- └─ PC -> `yield` 返回之后的位置
- └─ SP -> 当前协程 0 的栈顶

步骤 2：把 `scheduler.main_ctx` 恢复到 CPU

- └─ PC -> 调度器里上次保存的位置
- └─ SP -> 主线程自己的栈顶

恢复完成后：

- CPU 又回到主调度器继续跑；
- `task1` 的局部变量还留在协程 0 栈里；
- 下一次再 `resume(0)` 时，CPU 会重新加载 `coroutines[0].ctx` 中保存的 PC/SP，于是 `task1` 能从中断点继续。

可以把它压缩成一句话：

```
resume: 保存 main 的 PC/SP, 恢复 coroutine 的 PC/SP
yield : 保存 coroutine 的 PC/SP, 恢复 main 的 PC/SP
```

6. 进阶探讨：抢占式与非抢占式

- **非抢占（协作式）**：如果协程 A 进入死循环而不调用 `yield`，其他协程及主程序都会永久卡死。
- **抢占式**：主调度器可以利用 Linux 的 **时钟信号（SIGALRM）** 强制中断执行中的协程，强制执行上下文切换。这就是现代操作系统分时调度的缩影。

挑战 A 提示：抢占式调度

抢占式调度的目标是：即使协程没有主动调用 `yield()`，调度器也能在时间片用完时重新拿回控制权。

可以沿着下面的方向思考：

- 使用 `setitimer(ITIMER_REAL, ...)` 周期性产生 `SIGALRM`，例如每 50ms 触发一次。
- 使用 `sigaction(SIGALRM, ...)` 注册信号处理函数。
- 在信号处理函数中找到当前正在运行的协程，将它的状态从 `RUNNING` 改回 `SUSPEND` 或 `RUNNABLE`，再通过 `swapcontext(&co->ctx, &scheduler.main_ctx)` 切回主调度器。
- 增加一个类似 `in_scheduler` 的标志位：当程序正在调度器内部或正在执行上下文切换时，暂时不要响应抢占，避免状态被打断到一半。

这个挑战的难点不在 `setitimer` 本身，而在“什么时候可以安全抢占”。如果信号刚好发生在调度器修改 `current_id`、`state` 或执行 `swapcontext` 的中间，程序状态可能会被破坏。因此需要把调度器关键区域保护起来。

挑战 B 提示：具备时间感知的调度器

时间感知调度器的目标是：协程可以主动睡眠一段时间，在时间没到之前不参与调度。

可以沿着下面的方向思考：

- 在 `Coroutine` 控制块中增加一个字段，例如 `time_t wake_up_time`，记录该协程最早什么时候可以被恢复。
- 新增一种状态，例如 `SLEEPING`，表示协程不是结束了，而是在等待时间到达。
- 实现 `coroutine_sleep(int seconds)`：设置当前协程的 `wake_up_time = time(NULL) + seconds`，把状态改为 `SLEEPING`，然后切回 `main_ctx`。
- 主调度循环扫描协程表时，如果发现某个协程是 `SLEEPING`，就比较 `time(NULL)` 和 `wake_up_time`；时间到了再改回 `RUNNABLE`。

这个挑战的关键是区分 `SUSPEND` 和 `SLEEPING`：前者表示协程主动让出 CPU、下一轮可以继续运行；后者表示协程虽然让出了 CPU，但必须等到指定时间之后才能继续运行。